

Cours 3

Programmation orientée objet

Jérémy Kairis

Les instructions conditionnelles

Les instructions conditionnelles

Le bloc *if ... else*

En C#, les instructions conditionnelles permettent d'exécuter des blocs de code en fonction de l'évaluation de certaines conditions.

if: permet d'exécuter un bloc de code si une condition est vraie (true). Elle peut être suivie d'une instruction "***else***" pour gérer le cas où la condition n'est pas satisfaite.

else: L'instruction "***else***" est utilisée avec "***if***" pour spécifier un bloc de code à exécuter lorsque la condition du "***if***" n'est pas satisfaite.


```
var vrai = true;
```

```
if (vrai)
```

```
{
```

```
    Console.WriteLine("Vrai.");
```

```
}
```

```
else
```

```
{
```

```
    Console.WriteLine("Faux.");
```

```
}
```

Les instructions conditionnelles

La condition (...)

La condition doit renvoyer un Type de valeur “**bool**” et peut être le résultat d’un test sur des valeurs, d’une variable, le retour d’une méthode ou la combinaison de ces points.

Il y a 2 types d’opérateurs utilisés pour une condition :

- Les opérateurs sur les valeurs (“==”, “!=”, “<”, “<=”, “>”, “>=”)
- Les opérateurs logiques (“!”, “&&”, “||”)

```
var age = 33;  
var estCitoyen = true;  
bool peutVoter = age >= 18 && estCitoyen;  
  
if (peutVoter)  
{  
    Console.WriteLine("Vous pouvez voter.");  
}  
else  
{  
    Console.WriteLine("Désolé, vous ne pouvez pas voter.");  
}
```


Les instructions conditionnelles

Le bloc *if ... else if ... else*

Il arrive, cependant, que nous ayons plusieurs conditions et que nous ayons un traitement spécifique pour chacune d'entre elles.

Dans ce cas nous devons utiliser une variante du bloc “*if ... else*” à savoir “*if ... else if ... else*”.

else if: permet de tester une série de conditions les unes après les autres jusqu'à ce qu'une condition soit vraie. Cela permet de gérer plusieurs alternatives.

```
var nombre = 0;

if (nombre > 0)
{
    Console.WriteLine("Le nombre est positif.");
}
else if (nombre < 0)
{
    Console.WriteLine("Le nombre est négatif.");
}
else
{
    Console.WriteLine("Le nombre est nul.");
}
```


Les instructions conditionnelles

Le bloc *switch*

L’instruction “***switch***” est apte à sélectionner un branchement de l’exécution à partir d’un jeu de branchement mutuellement exclusifs.

Cependant, lorsqu’un “***case***” contient du code, ce dernier doit se terminer par le mot clé “***break***”.

Les “***case***” peuvent être regroupés ensemble pour n’écrire qu’une fois le code, mais la règle précédente s’applique toujours.

```
var numeroJour = int.Parse(Console.ReadLine());
```

```
var nomJour = string.Empty;
```

```
switch (numeroJour)
```

```
{
```

```
    case 1:
```

```
        nomJour = "Lundi";
```

```
        break;
```

```
    case 2:
```

```
        nomJour = "Mardi";
```

```
        break;
```

```
    ...
```

```
    default:
```

```
        nomJour = "Jour invalide";
```

```
        break;
```

```
}
```

...

case 6:

case 7:

nomJour = "Week-end";

break;

...

Les instructions conditionnelles

Exercices

1. Demander à l'utilisateur d'encoder 1 nombre (*int*), si le modulo de 2 est 0, afficher "le nombre est paire" sinon "le nombre est impaire" et gérer les erreurs.
2. Développez une calculatrice simple. L'utilisateur devrait pouvoir entrer deux nombres et un opérateur (+, -, *, /) pour effectuer une opération arithmétique. Le programme devrait utiliser l'instruction "**switch**" pour déterminer quelle opération effectuer en fonction de l'opérateur saisi et afficher le résultat.

```
Console.WriteLine("Entrez chaque nombre suivi de 'Enter'");
```

```
if (decimal.TryParse(Console.ReadLine(), out var a)  
    && decimal.TryParse(Console.ReadLine(), out var b))
```

```
{
```

```
    Console.WriteLine("Entrez l'opérateur (+,-,*,/)");
```

```
    string? operateur = Console.ReadLine();
```

```
    var result = operateur switch
```

```
    {
```

```
        "+" => a + b,
```

```
        "-" => a - b,
```

```
        "*" => a * b,
```

```
        "/" => a / b,
```

```
        _ => throw new InvalidOperationException(),
```

```
    };
```

```
    Console.WriteLine($"Résultat: {result}");
```

```
}
```

```
else
```

```
{
```

```
    Console.WriteLine("Nombre invalide");
```

```
}
```

Les opérateurs

Les opérateurs

L'opérateur ternaire ? :

L'opérateur ternaire “?” est une forme abrégée de la construction “*if ... else*” optimisée pour l'affectation de valeur. Il tient son nom du fait qu'il implique 3 opérandes :

- La condition
- La valeur si vrai
- La valeur si faux

Opérateur ternaire = (condition) ? valeur si vrai : valeur si faux;

```
var nombre = 7;  
var parite = nombre % 2 == 0  
    ? "pair"  
    : "impair";
```

```
Console.WriteLine($"Le nombre {nombre} est {parite}.");
```


Les opérateurs

Pré & post incrémentation ou décrémentation ++ --

La différence entre la pré et la post incrémentation ou décrémentation est définie sur le moment où la variable va changer de valeur.

Dans le cadre de la pré-incrémentation (**++x**) ou de la pré-décrémentation (**--x**), **x** changera de valeur avant d'avoir été utilisée.

Dans le cadre de la post-incrémentation (**x++**) ou de la post-décrémentation (**x--**), **x** changera de valeur après avoir été utilisée.

```
int x = 5;
```

```
int y = 5;
```

```
int result1 = ++x;
```

```
Console.WriteLine("Pré-incrémentation (++x): x=" + x + ", résultat1=" + result1);
```

```
int result2 = --y;
```

```
Console.WriteLine("Pré-décrémentation (--y): y=" + y + ", résultat2=" + result2);
```

```
int a = 5;
```

```
int b = 5;
```

```
int result1 = a++;
```

```
Console.WriteLine("Post-incrémentation (a++): a=" + a + ", résultat1=" + result1);
```

```
int result2 = b--;
```

```
Console.WriteLine("Post-décrémentation (b--): b=" + b + ", résultat2=" + result2);
```


Les opérateurs

L'opérateur coalesce ??

L'opérateur “**??**” est appelé l'opérateur coalesce.

Ce dernier retourne l'opérande de partie gauche si ce dernier n'est pas “***null***” et dans le cas contraire, il retourne l'opérande de partie droite.

Cet opérateur peut-être répété.

La version “**??=**” existe aussi et permet d'assigner une valeur si la variable est “***null***”.

```
int? valeurNullable = null;
```

```
int valeurParDefaut = 10;
```

```
int resultat = valeurNullable ?? valeurParDefaut;
```

```
Console.WriteLine("Le résultat est : " + resultat);
```

Les opérateurs

L'opérateur de type *typeof*

L'opérateur “***typeof***” retourne le “***Type***” d'un type spécifié. Il permet d'obtenir des informations sur les types au moment de la compilation, ce qui peut être utile pour effectuer des vérifications de type, pour gérer des scénarios de réflexion et pour de nombreuses autres tâches de programmation.

Cependant, il ne peut pas être utilisé pour obtenir des informations sur les types d'objets spécifiques à l'exécution, pour cela, la méthode “***GetType()***” est utilisée.


```
int entier = 42;  
object objet = entier;
```

```
Type typeStatique = typeof(int);  
Type typeDynamique = objet.GetType();
```

```
Console.WriteLine($"Type statique de 'entier': {typeStatique}");  
Console.WriteLine($"Type dynamique de 'objet': {typeDynamique}");
```

Les opérateurs

L'opérateur de contrôle de type *is*

L'opérateur “*is*” contrôle si une variable est compatible avec un type donné.

Il vérifie si un objet est une instance d'un certain type ou d'une classe spécifique. Il renvoie un booléen, vrai (***true***) si l'objet est du type spécifié, sinon il renvoie faux (***false***).


```
object o = "Bonjour";

if (o is string)
{
    Console.WriteLine("La variable 'o' est une chaîne de caractères.");
}
else
{
    Console.WriteLine("La variable 'o' n'est pas une chaîne de caractères.");
}
```

Les opérateurs

L'opérateur de conversion *as*

L'opérateur “*as*” converti explicitement le résultat pour donner une valeur d'un type donné ou “*null*” s'il ne parvient pas à convertir.

De ce fait, cet opérateur ne peut être utilisé qu'avec des types références comme “*Nullable<T>*”.

Contrairement à l'opérateur de conversion classique, “*(int)x*”, l'opérateur “*as*” ne déclenche jamais d'erreur et l'évaluation ne se fera qu'une seule fois.

```
object texte = "123";
```

```
object nombre = 42;
```

```
var x = texte as int?;
```

```
var y = nombre as int?;
```

```
Console.WriteLine(x == null ? "texte n'est pas un int" : $"x = {x}");
```

```
Console.WriteLine(y == null ? "nombre n'est pas un int" : $"y = {y}");
```

```
Console.WriteLine((int?)texte);
```


Les opérateurs

L'opérateur de plage `..` [^]

L'opérateur de plage (range operator) `..` est utilisé pour spécifier une plage, et il peut être utilisé à la fois dans des indices positifs et négatifs.

Par exemple :

`0..5` représente la plage de 0 à 4 (les indices 0, 1, 2, 3, 4 inclus).

`1..^1` représente la plage du deuxième élément (indice 1) à l'avant-dernier élément (indice -2).

```
int[] nombres = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100];
```

```
var plage1 = nombres[0..5];  
var plage2 = nombres[1..^1];
```

```
Console.WriteLine("Plage 1 :");  
foreach (var nombre in plage1)  
    Console.WriteLine(nombre);
```

```
Console.WriteLine("\nPlage 2 :");  
foreach (var nombre in plage2)  
    Console.WriteLine(nombre);
```

Les opérateurs

L'ordre de priorité

*() . [] x++ x- new typeof * / % + - < <= > >= is as == != && // ?: = += -= *= /= %=*

Les parenthèses permettent toujours de s'assurer de l'ordre d'exécution.


```
int a = 5, b = 3, c = 7, result;
```

```
// Utilisation de parenthèses pour contrôler l'ordre d'exécution
```

```
result = (a + b) * c; // (5 + 3) * 7 = 56
```

```
Console.WriteLine($"Résultat avec parenthèses : {result}");
```

Les opérateurs

Exercice

1. Vérifier qu'un compte bancaire BBAN est valide et afficher le résultat dans la console. (Il est valide si le modulo des 10 premiers chiffres par 97 donne les 2 derniers ou ils valent 97 s'il vaut 0.)

Exemple valide : 539007547034.


```
Console.WriteLine("Entrez BBAN");  
var bban = Console.ReadLine();  
Console.WriteLine(  
    bban?.Length == 12  
    && long.TryParse(bban[..10], out var num)  
    && ((num % 97).ToString("00") == num.ToString()  
        || (num % 97 == 0 && num.ToString() == "97"))  
    ? "Valide"  
    : "Invalide");
```

Les boucles

Les boucles

Présentation

C# propose 4 constructions itératives afin de répéter les actions, que le nombre de fois soit connu ou non.

Il s'agit des boucles “***for***”, “***while***”, “***do ... while***” et du “***foreach***”.

Les boucles

La boucle *for*

La boucle `for` est le plus souvent utilisée lorsque nous connaissons le nombre d'itérations à effectuer.

Cette boucle est composée de trois éléments :

1. L'initialiseur : l'expression évaluée avant l'exécution de la première boucle.
2. La condition : l'expression vérifiée avant chaque nouvelle itération de la boucle. (Elle doit être égale à “***true***” pour qu'une nouvelle itération puisse être effectuée)
3. L'itérateur : une expression qui est évaluée après chaque itération. Ces itérations cessent quand la condition renvoi “***false***”.

```
for (int i = 1; i <= 5; i++)  
{  
    Console.WriteLine($"Itération {i}");  
}
```

Les boucles

La boucle *while*

Le principe de la boucle “***while***” est que :

Tant que la condition est “***true***”, le bloc d’instructions s’exécutera, la condition est vérifiée avant l’itération.


```
int compteur = 1;

while (compteur <= 5)
{
    Console.WriteLine($"Itération {compteur}");
    compteur++;
}
```

Les boucles

La boucle *do ... while*

La boucle “***do ... while***” fonctionne dans le même principe que la boucle “***while***” à ceci près que la condition est vérifiée après le traitement.

Ce qui a pour effet, d'imposer à la boucle, d'effectuer au moins une fois le traitement.


```
int compteur = 1;  
  
do  
{  
    Console.WriteLine($"Itération {compteur}");  
    compteur++;  
} while (compteur <= 5);
```

Les boucles

La boucle *foreach*

La boucle “***foreach***” est le plus souvent utilisée lorsque nous souhaitons boucler sur l’entièreté des éléments d’une collection.

Elle va donc parcourir un par un les éléments en stockant l’élément courant dans une variable définie durant de l’itération.

Lors du traitement d’une boucle “***foreach***” il est interdit de modifier le nombre d’éléments de la collection.

```
string[] fruits = ["Pomme", "Banane", "Orange"];
```

```
foreach (string fruit in fruits)  
{  
    Console.WriteLine(fruit);  
}
```


Les boucles

Exercices

1. Calculer la somme de tous les nombres pairs de 1 à 100 à l'aide d'une boucle “***for***”.
2. Calculer la factoriel d'un nombre entré au clavier à l'aide d'une boucle “***while***”.
3. Demander à l'utilisateur de saisir un nombre entier et afficher la table de multiplication de ce nombre de 1 à 10.

Les tableaux

Les tableaux

À une dimension

Un tableau représente un nombre fixe de variables (appelés éléments) d'un type défini.

Les éléments du tableau sont stockés de manière contigüe en mémoire, ce qui fournit un accès hautement efficient.

Pour déclarer un tableau nous utiliserons les “**[]**” à coté du type de la variable que nous déclarerons. Une fois instancié, chaque élément se verra attribué, par défaut, la valeur de “***default(T)***”.

Le premier élément du tableau à pour index **0**.

Il est à noter également, derrière ***int[]*** se cache une instanciation de la classe “***Array***” ce qui fait de notre “***int[]***” un type référence


```
int[] tableauEntiers = new int[2];
```

```
tableauEntiers[0] = 10;
```

```
tableauEntiers[1] = 20;
```

```
Console.WriteLine("Éléments du tableau :");
```

```
Console.WriteLine("Élément 0 : " + tableauEntiers[0]);
```

```
Console.WriteLine("Élément 1 : " + tableauEntiers[1]);
```

```
int longueur = tableauEntiers.Length;
```

```
Console.WriteLine("Longueur du tableau : " + longueur);
```

Les tableaux

Multidimensionnels

C# gère 2 types de tableaux multidimensionnels :

1. Les tableaux matriciels “[,]” : où chaque dimension contient le même nombre d’éléments.
2. Les tableaux orthogonaux “[][]” : où chaque ligne peut contenir un nombre différent d’éléments.

Les tableaux matriciels sont plus adaptés lorsque toutes les dimensions doivent avoir la même taille, tandis que les tableaux orthogonaux sont utiles pour les structures irrégulières.

Les tableaux

Exercices

1. Calculer la somme et la moyenne des éléments d'un tableau
2. Trouver les éléments maximum et minimum dans un tableau
3. Remplir et afficher une matrice de 10x10 avec les nombres de 1 à 100

```
var lignes = 10;  
var colonnes = 10;  
var matrice = new int[lignes, colonnes];
```

```
var n = 1;  
for (int i = 0; i < lignes; i++)  
{  
    for (int j = 0; j < colonnes; j++)  
    {  
        matrice[i, j] = n++;  
        Console.Write($"{matrice[i, j],3} ");  
    }  
    Console.WriteLine();  
}
```

Les méthodes

Les méthodes

Fonctions

Les méthodes en C# sont des blocs de code nommés qui effectuent des tâches spécifiques. Elles sont définies avec un nom, un type de retour (ce qu'elles renvoient, le cas échéant), et des paramètres (les données d'entrée).

Les méthodes peuvent être appelées à partir d'autres parties du code pour exécuter leur fonction. Il existe deux types principaux de méthodes : statiques (appartenant à la classe) et d'instance (associées à une instance de classe).

Les méthodes sont essentielles pour organiser et réutiliser du code, améliorant ainsi la lisibilité et la maintenance.

Les méthodes

void

Lorsqu'une méthode a le type de retour ***void***, cela signifie qu'elle ne renvoie aucune valeur.

En d'autres termes, elle effectue des opérations sans produire de résultat exploitable.

```
void DireBonjour()  
{  
    Console.WriteLine("Bonjour");  
}
```


Les méthodes

return

Le mot-clé "***return***" est utilisé pour renvoyer une valeur depuis une méthode vers l'endroit où elle a été appelée.

Il met fin à l'exécution de la méthode en cours et transmet la valeur spécifiée, qui peut être utilisée ou stockée pour d'autres opérations.

Les méthodes avec un type de retour autre que "***void***" doivent inclure une instruction "***return***" pour renvoyer une valeur correspondante à leur type de retour.

Les méthodes d'une instruction peuvent aussi retourner le résultat avec une flèche "***=>***".

```
int Additionner(int a, int b)
{
    return a + b;
}
```

// ou

```
int Additionner(int a, int b) => a + b;
```


Les méthodes

Invocation

Les méthodes sont invoquées par leur nom suivi de “**()**” et en y passant les paramètres nécessaires.

```
void DireBonjour() => Console.WriteLine("Bonjour");
```

```
int Additionner(int a, int b) => a + b;
```

```
DireBonjour();
```

```
var somme = Additionner(1, 2);
```

Les méthodes

Les paramètres

La plupart des méthodes doivent recevoir des éléments afin de fonctionner correctement, ces éléments sont appelé des paramètres.

Lors de la déclaration des paramètres, nous devons spécifier le type et leur donner un nom que nous pourrons utiliser dans la méthode.

Ensuite, lors de l'appel de nous passerons les valeurs de ses paramètres afin que la méthode effectue son traitement.


```
void AfficherInfosPersonne(string nom, int age, bool estEtudiant)
{
    Console.WriteLine("Nom : " + nom);
    Console.WriteLine("Âge : " + age);
    Console.WriteLine("Est étudiant : " + estEtudiant);
}
```

```
AfficherInfosPersonne("John", 18, true);
```

Les méthodes

Les paramètres facultatifs

Les paramètres ayant une valeur par défaut sont appelés paramètres facultatifs.

La valeur par défaut est définie à la déclaration du paramètre à l'aide de l'opérateur “=”.

Cependant, les paramètres facultatifs doivent être les derniers paramètres à être définis.

```
void AfficherInfosPersonne(string nom, int age = 18, bool estEtudiant = true)
{
    Console.WriteLine("Nom : " + nom);
    Console.WriteLine("Âge : " + age);
    Console.WriteLine("Est étudiant : " + estEtudiant);
}
```

```
AfficherInfosPersonne("John");
```


Les méthodes

params

Lorsque le nombre de paramètres à passer à une fonction inconnu, on peut utiliser le mot clé “*params*”.

3 règles sont à respecter :

- Il ne peut y en avoir qu’un
- Il doit être le dernier paramètre
- Toutes les valeurs doivent être du même type que celui déclaré

```
int AdditionnerNombres(params int[] nombres)
{
    var somme = 0;
    foreach (int nombre in nombres)
    {
        somme += nombre;
    }
    return somme;
}
```

```
var resultat = AdditionnerNombres(1, 2, 3, 4, 5);
Console.WriteLine(resultat);
// ou
Console.WriteLine(AdditionnerNombres(1, 2, 3, 4, 5));
```


Les méthodes

La surcharge

La signature d'une méthode est définie par son nom ainsi que le nombre et le type de(s) paramètre(s) passé(s) en entrée.

La surcharge de méthode est un principe qui permet de déclarer plusieurs méthodes ayant le même nom pour peu que le reste de leur signature soit différent.

2 différences sont possibles pour la surcharge :

- Le type des paramètres
- Le nombre de paramètre

```
int Additionner(int a, int b)
```

```
=> a + b;
```

```
int Additionner(int a, int b, int c)
```

```
=> a + b + c;
```

```
double Additionner(double a, double b)
```

```
=> a + b;
```

```
double Additionner(double a, double b, double c)
```

```
=> a + b + c;
```

Les méthodes

Exercices

1. Écrivez une fonction nommée "AdditionnerNombresPairs" qui prend un entier positif comme argument et calcule la somme de tous les nombres pairs de 1 à cet entier.
2. Écrivez une fonction nommée "EstPalindrome" qui prend une chaîne de caractères en entrée et détermine si elle est un palindrome.
3. Écrivez une fonction nommée "ConvertirCelsiusEnFahrenheit" qui prend une température en degrés Celsius en entrée et la convertit en degrés Fahrenheit.

Jeux

Le marché

Créez un jeu de gestion de panier de fruits au marché à l'aide d'un tableau de chaînes de caractères.

L'utilisateur peut ajouter maximum 5 fruits, les retirer, les afficher et les rechercher.

Gérez les doublons lors de l'ajout.

Permettez à l'utilisateur de quitter le jeu via le menu.

Github

Le marché

- Déposer le projet sur votre compte Github en public
- M'envoyer le lien de votre Github via Teams